MTA

AUDIO EDITOR

Manuale Amministratore

Deployment, sicurezza, storage e operations



MTA Audio Editor 0.3.0-r210

Build: 20261015171900

Alessandro De Salvo

braket71@gmail.com

https://github.com/desalvo/mta-audio-editor

EUPL-1.2

Digital Audio Workstation con supporto MTA

Windows macOS Linux Android iOS

Guida operativa per installazione, sicurezza, storage, backup, client desktop/mobile, pipeline audio, formato MTA, osservabilità e release.

Indice

1. Architettura
2. Deployment
3. Configurazione
4. Storage e backup
5. Sicurezza e utenti
6. Pipeline audio
7. Client desktop e mobili
8. Formato MTA
9. Operazioni e osservabilità
10. Upgrade e rollback
11. CI/CD e release
12. Troubleshooting amministrativo
13. Demucs locale iPhone/iPad - provisioning Core ML
14. Aggiornamenti nativi e modelli Demucs
15. Aggiornamento automatico modelli Demucs mobile
16. Registro modelli stem esteso
17. Gestione dinamica dei modelli Demucs
18. Token runtime per i modelli
19. Motori Lyrics/Chords e MP4
20. Integrazione repertori SIAE / Soundreef
21. Gestione operativa r210

Appendici tecniche

22. Checkpoint distribuzione MIDI/GM e moduli AI
23. SAM Audio: accesso e diagnosi del servizio

1. Architettura

MTA Audio Editor usa un backend FastAPI e un frontend web condiviso da browser, client desktop e app mobili. Rendering, waveform, Demucs e generazione MTA sono operazioni backend.

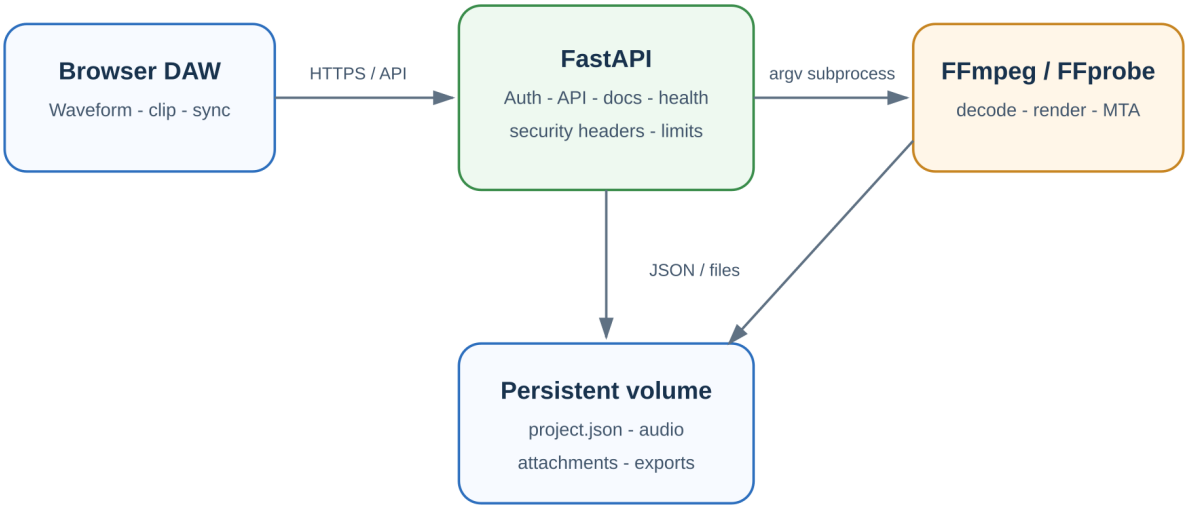


Figura 1 - Client, API, processing e storage.

Componente	Piattaforme

Server	Linux, Docker, Kubernetes
Desktop	Windows, macOS Intel, macOS Apple Silicon
Mobile	Android, iOS/iPadOS
Browser	Browser moderni con Web Audio API

2. Deployment

Per Docker Compose usare un volume persistente per `/data/projects`. In Kubernetes usare il PVC previsto dal manifest e un Ingress opzionale. Il limite upload applicativo e quello dell'Ingress devono essere coerenti.

```
docker compose up -d
# Kubernetes
kubectl apply -k k8s/overlays/nginx
```

TLS: in produzione terminare HTTPS su un reverse proxy/Ingress affidabile. Configurare una URL pubblica coerente per link email e OAuth quando utilizzati.

3. Configurazione

Le impostazioni principali sono fornite tramite variabili d'ambiente e Secret. Tra i gruppi da gestire: bootstrap amministratore, upload massimo, SMTP, OAuth, sicurezza sessione e storage.

Area	Esempi	Nota
Bootstrap	MTA_ADMIN_USERNAME, MTA_ADMIN_PASSWORD, MTA_ADMIN_EMAIL	Necessari per il primo amministratore
Upload	MTA_MAX_UPLOAD_MB	Allineare Ingress e applicazione
Pubblicazione	MTA_PUBLIC_URL	Consigliato dietro proxy per callback/link
SMTP	host, porta, TLS, credenziali	Secret, non ConfigMap pubblica

4. Storage e backup

Il root persistente contiene database utenti/configurazione e directory progetto. Ogni progetto mantiene modello, audio, attachment, originali, cache e analisi. I backup devono includere l'intero root persistente quando si vuole un ripristino completo.

Strategia: snapshot coerente del volume, retention su più generazioni e test periodico di restore. Per Kubernetes verificare che StorageClass e CSI supportino correttamente UID/GID e fsGroup usati dal pod.

5. Sicurezza e utenti

Le installazioni server supportano account locali, verifica email, approvazione amministrativa, TOTP e provider OAuth opzionali. Le funzioni social sono disabilitate per default. Le richieste OAuth sono vincolate a HTTPS e host provider consentiti.

La modalità desktop native single-user evita database account e schermate login, ma mantiene i controlli di accesso interni necessari alle API locali. Le credenziali e i token non devono essere incorporati nelle immagini container.

6. Pipeline audio

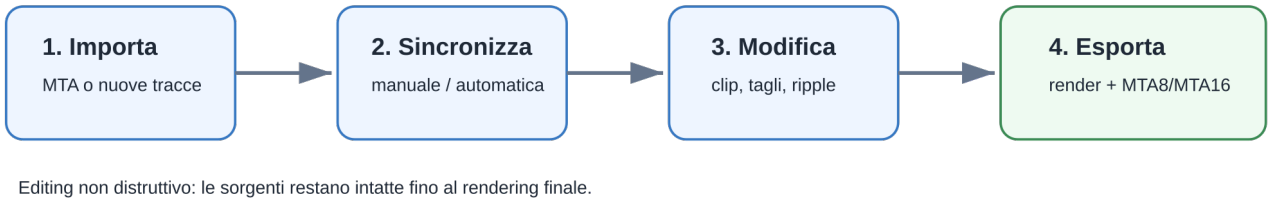


Figura 2 - Editing non distruttivo e rendering.

Le operazioni FFmpeg e Demucs possono essere CPU/GPU intensive. Le preview traccia vengono cacheate con una firma che include sorgente, stato traccia e parametri rilevanti. I job lunghi espongono avanzamento e risultato tramite API.

6.1 Preview dinamica e Render

La preview dinamica bufferizza le tracce sul client e mantiene un clock di sincronizzazione. Render genera un master server-side. Con i meter RealTime, un ramo monitor delle tracce alimenta gli analyser senza diventare udibile.

7. Client desktop e mobili

I desktop includono backend locale e selettori filesystem nativi. Le app mobili sono client del backend remoto.

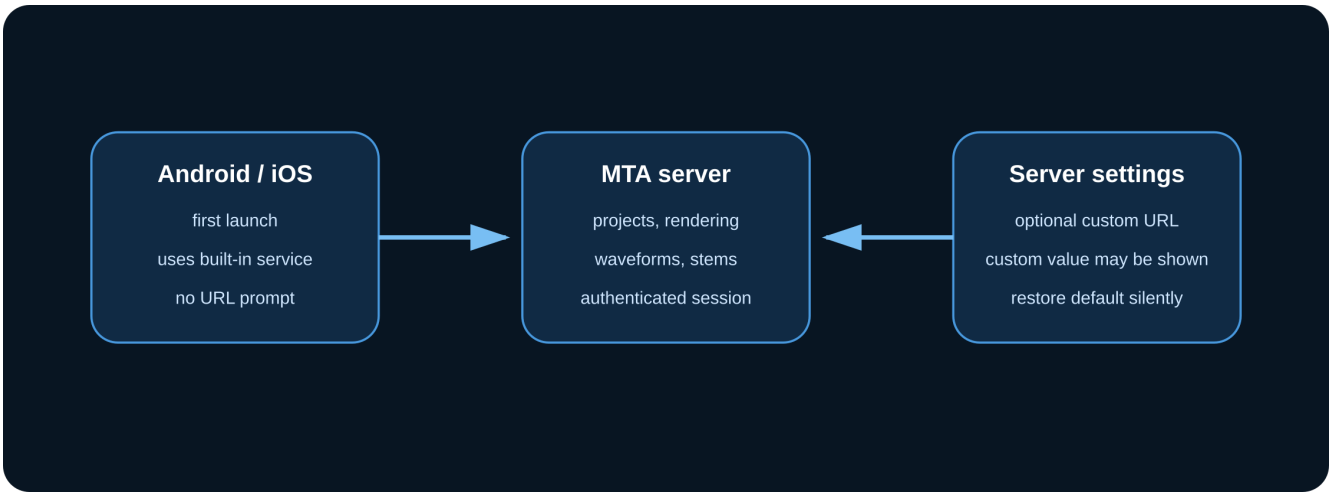


Figura 3 - Connessione mobile.

Android/iOS usano automaticamente il servizio preconfigurato senza prompt durante installazione/primo avvio. L'indirizzo predefinito resta interno e non viene mostrato. Un URL personalizzato può essere configurato successivamente e, poiché è scelto dall'utente, può essere mostrato nelle impostazioni.

8. Formato MTA

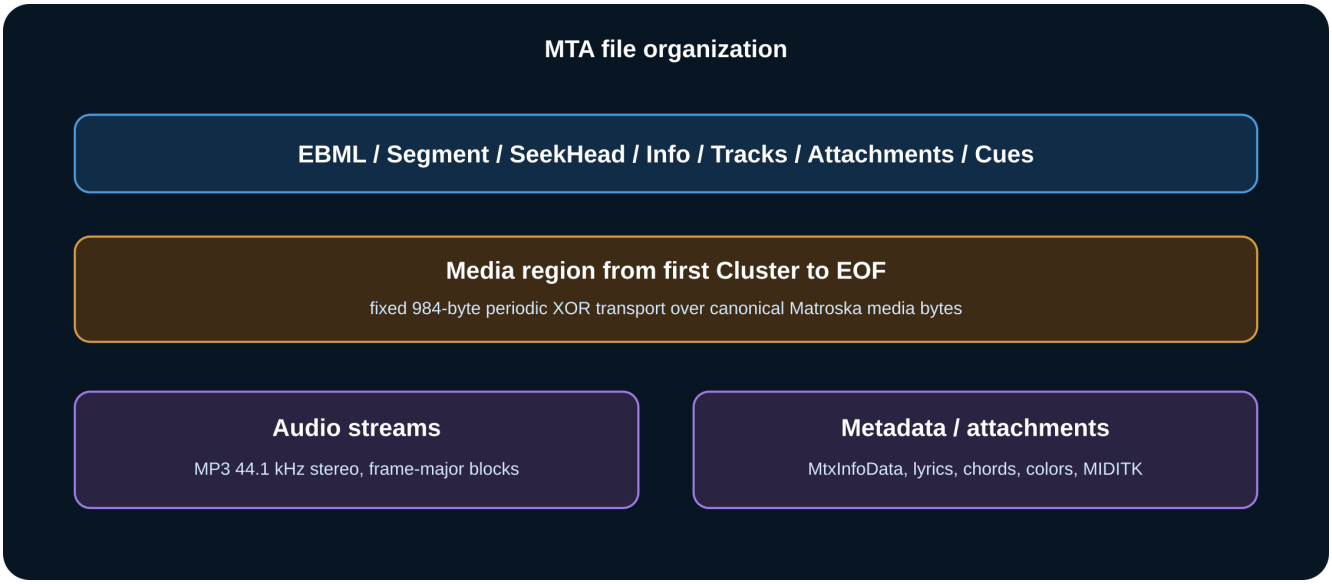


Figura 4 - Struttura generale del container MTA.

MTA usa strutture EBML/Matroska, stream audio MP3 e attachment di metadati. La regione media a partire dal primo Cluster usa un trasporto XOR periodico documentato. La specifica completa IT/EN è in `docs/MTA_FORMAT_FINAL_SPEC_IT.md` e `docs/MTA_FORMAT_FINAL_SPEC_EN.md`.

8.1 Profili Click/Melody

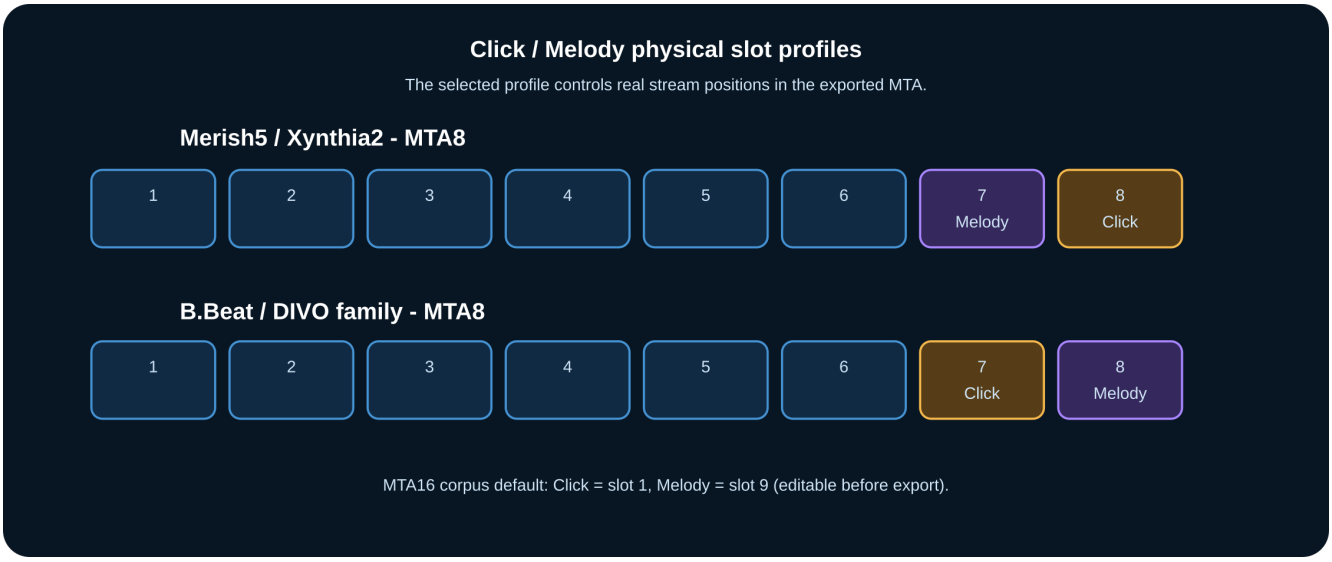


Figura 5 - Profili di export MTA8/MTA16.

I profili MTA8 preservano posizioni fisiche specifiche. Il default MTA16 basato sul corpus usa Click 1 e Melody 9, sempre modificabili. Gli slot silenziosi intermedi vengono creati quando necessari per mantenere il numero fisico dello stream.

9. Operazioni e osservabilità

Monitorare health endpoint, spazio persistente, tempi di render, job lunghi, log applicativi, FFmpeg/Demucs e disponibilità del servizio. Le probe Kubernetes devono rimanere leggere e indipendenti dai job audio.

Segnale	Controllo
Health	`/api/health`
Storage	Spazio libero e inode del volume
Job	Durata, stato, errori server-side
Auth	Errori login/OAuth/SMTP
Media	ffmpeg, Demucs, generazione waveform/export

10. Upgrade e rollback

Prima dell'upgrade effettuare backup, verificare release notes e production gate, aggiornare immagini/manifests e validare l'apertura di progetti esistenti. Per rollback ripristinare la versione applicativa e, se necessario, lo snapshot del volume coerente con quella versione.

11. CI/CD e release

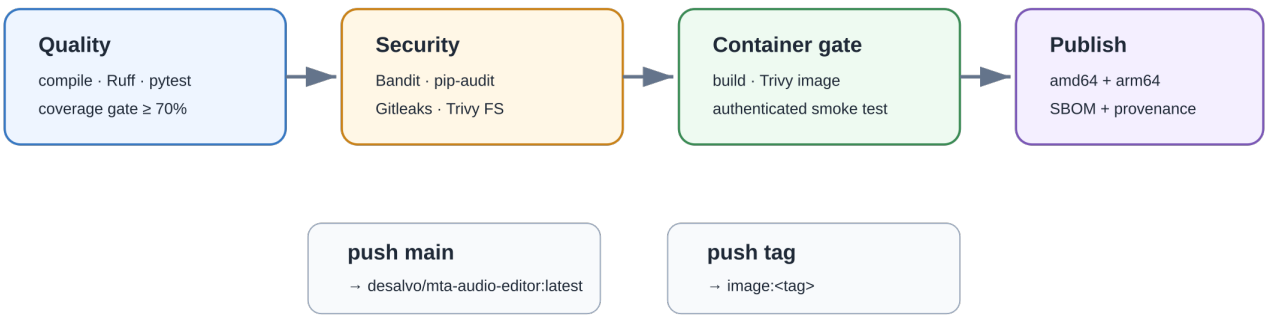


Figura 6 - Gate di qualità e packaging.

Il production gate include lint, test, controlli sicurezza e packaging. Le build mobile e native sono prodotte da workflow separati/step dedicati; i secret di signing restano nel sistema CI.

Identità di release: `VERSION` contiene la versione pubblica del prodotto (es. `0.2.0`), `REVISION` identifica la revisione esatta del sorgente/package (es. `154`) e `BUILD_INFO` resta l'identificativo di build indipendente a 14 cifre. I tag stabili usano `v0.2.0`; gli artefatti possono includere `-r154`. Gli eventuali versionCode/CFBundleVersion richiesti dalle piattaforme sono codici tecnici di packaging e non sostituiscono `BUILD_INFO`.

12. Troubleshooting amministrativo

Sintomo	Verifica
Upload 413/limitato	Allineare limite app e proxy/Ingress
Job stems molto lento	CPU/GPU, RAM, spazio temporaneo, modello Demucs
Login/email non funzionano	SMTP, URL pubblica, log auth, stato utente
Pod non scrive sul PVC	permessi storage, UID/GID, fsGroup
Client mobile non collega server custom	URL custom, TLS, DNS e raggiungibilità dal dispositivo
MTA non accettato dall'hardware	profilo slot, target, firmware e validazione sul dispositivo

13. Demucs locale iPhone/iPad - provisioning Core ML

Il server può distribuire modelli Demucs compatibili con Core ML impostando `MTA_DEMUCS_COREML_MODEL_DIR` su una directory contenente `demucs-2.mlmodel`, `demucs-4.mlmodel`, `demucs-6.mlmodel` e/o `demucs-8.mlmodel`. Il client iOS/iPadOS scarica il modello autenticandosi con la

sessione corrente, lo compila localmente e lo conserva in Application Support. Le separazioni successive possono funzionare offline sul dispositivo; in modalità Auto il server resta il fallback per modelli mancanti, brani oltre il limite locale o errori di inferenza.

14. Aggiornamenti nativi e modelli Demucs

Le applicazioni native Windows, macOS, Android, iPhone e iPad controllano GitHub per nuove versioni. Il canale **Stable** usa solo release/tag, mentre **Early release** include anche la prerelease `early-main` prodotta dall'ultima build valida di main.

Su iPhone/iPad l'app rende disponibile automaticamente un modello Demucs Core ML 4-stem predefinito: usa prima l'eventuale modello bundled e, se assente, lo scarica dal server configurato. Gli hash SHA-256 vengono poi confrontati con il catalogo server e i modelli installati vengono aggiornati quando cambia il fingerprint. Il modello locale già installato resta utilizzabile offline.

Su iOS/iPadOS l'aggiornamento dell'app deve comunque passare da TestFlight/App Store o da un canale di distribuzione autorizzato dal sistema operativo.

15. Aggiornamento automatico modelli Demucs mobile

Il server Docker/Kubernetes aggiorna periodicamente i repository Core ML (iPhone/iPad) e ONNX (Android) da un manifest HTTPS, verificando SHA-256 e sostituendo i file in modo atomico. I client mobili controllano il catalogo ogni 6 ore e all’avvio. La preferenza **Update solo con Wi-Fi** è attiva per default: gli aggiornamenti periodici dei modelli vengono scaricati solo su Wi-Fi, salvo disabilitazione esplicita dello switch. Il modello baseline 4-stem viene invece sempre predisposto alla prima occasione di rete disponibile.

16. Registro modelli stem esteso

Configurare modelli server aggiuntivi con `MTA_DEMUCS_MODEL_REGISTRY` o `MTA_DEMUCS_MODEL_REGISTRY_FILE`. Ogni profilo dichiara `model`, `stem_count`, `stem_labels` e opzionalmente `engine`/`display_name`. I repository Core ML/ONNX vengono scoperti dinamicamente tramite file `demucs-N`, fino a 64 stem.

17. Gestione dinamica dei modelli Demucs

Il server mantiene periodicamente aggiornato l'intero catalogo di modelli configurato, inclusi i modelli Demucs ufficiali disponibili nel runtime e gli artefatti Core ML/ONNX pubblicati dal manifest. I client mobile e desktop scaricano on-demand il modello scelto quando viene richiesto lo splitting. Dalle opzioni locali è possibile cancellare un modello, riscaricarlo o forzarne l'aggiornamento. L'amministratore può blacklistare un modello: il modello viene rimosso dallo storage gestito e scompare dal catalogo disponibile a web e client finché non viene rimosso dalla blacklist.

18. Token runtime per i modelli

`POST /api/models/token` emette token Bearer revocabili con solo scope `models:read`. I download Core ML, ONNX e bundle nativi richiedono sessione autenticata o tale token. La CI usa asset della release GitHub `demucs-models` e non credenziali runtime.

19. Motori Lyrics/Chords e MP4

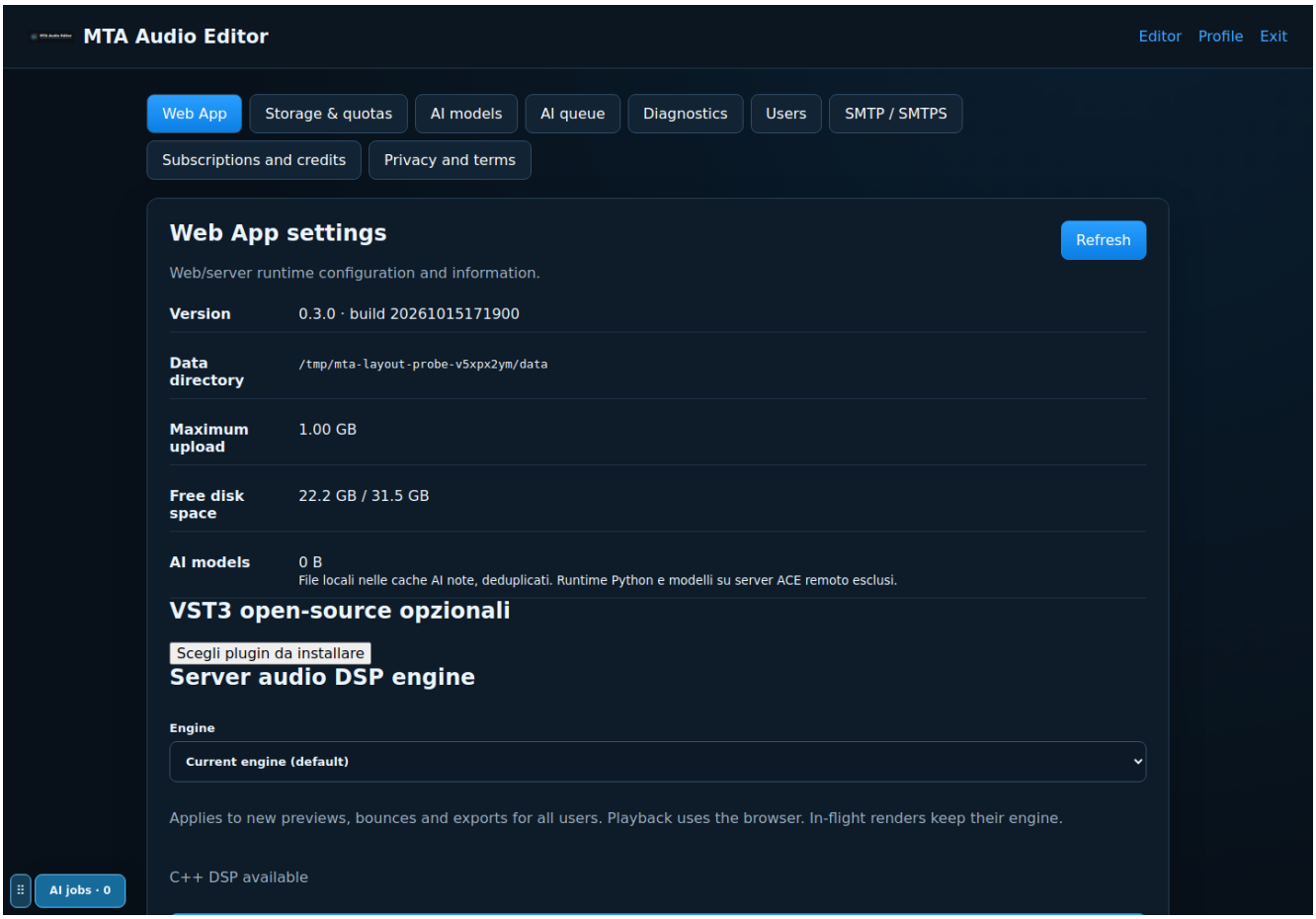
Per la massima accuratezza delle lyrics, il default è `MTA_LYRICS_WHISPER_MODEL=large-v3`; è configurabile anche `MTA_LYRICS_BEAM_SIZE`; con temperatura 0 viene usata una ricerca beam deterministica con patience aumentata. Per i chords, se Sonic Annotator con plugin Chordino/NNLS-Chroma è installato viene preferito automaticamente. L'export MP4 richiede FFmpeg con filtro `ass`/libass e codec H.264/AAC.

20. Integrazione repertori SIAE / Soundreef

SIAE e Soundreef sono i provider attivi per default nella ricerca repertorio. Le pagine pubbliche ufficiali restano sempre disponibili come fallback. Per una ricerca strutturata server-to-server configurare endpoint HTTPS autorizzati/licenziati con `MTA_RIGHTS_SIAE_SEARCH_URL` e/o `MTA_RIGHTS_SOUNDREEF_SEARCH_URL`. L’endpoint deve restituire JSON con una lista in `results`, `items`, `works` o `data`; vengono riconosciuti campi comuni quali title/original_title, authors/writers/composers, performers/artists, publishers/editors, ISWC, IPI e work_id. L’app non presume endpoint privati o non documentati delle collecting.

21. Gestione operativa r210

Impostazioni raccoglie Web App, utenti, diagnostica, SMTP, sicurezza, SSO e gestione delle risorse. La ricerca utenti trova dinamicamente nome, username o email. SMTP include una mail di prova: verificare recapito prima di affidarsi alle notifiche. Google, Facebook e GitHub restano disattivati finché configurati; l’identità SSO non sostituisce l’approvazione admin. Le nuove registrazioni richiedono verifica email, secondo fattore e approvazione secondo il flusso prescelto.

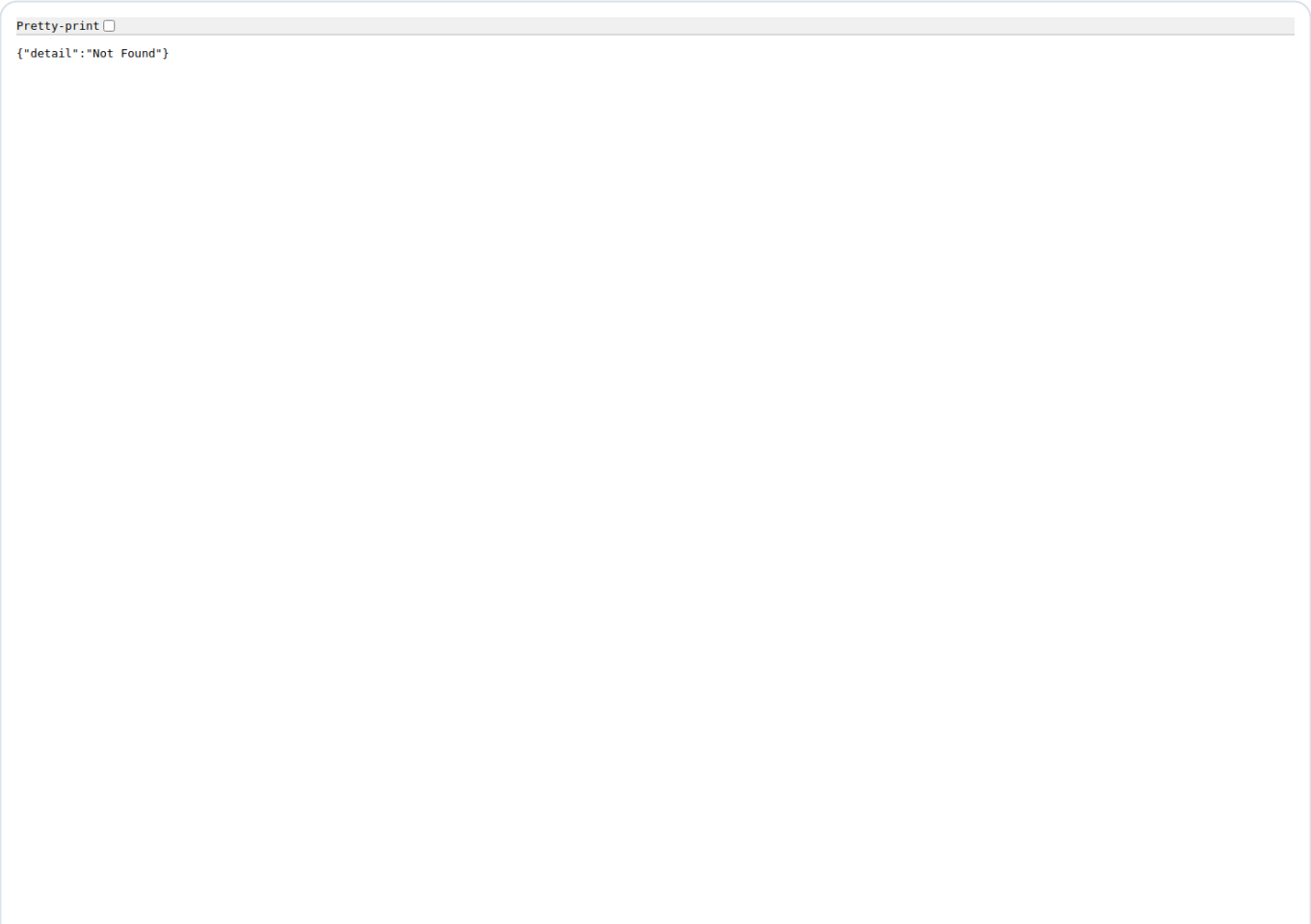


Interfaccia amministrativa su un server di prova senza dati personali.

Quote, piani, crediti e pagamenti

Le quote CPU/GPU/storage e le tariffe sono configurabili. I nuovi utenti hanno 5 GiB di storage; gli admin non hanno quota per default e non possono essere disabilitati o eliminati. Un utente senza abbonamenti attivabili mantiene l’accounting, senza billing né crediti. I crediti richiedono l’abilitazione globale e un abbonamento attivo/promo; gli admin possono assegnare promo e crediti manualmente. Le modifiche ai limiti della sottoscrizione attiva non azzerano il periodo né i consumi: per gli aumenti scegliere se gratuiti o a pagamento.

La cessazione può essere immediata; il canone viene proporzionato se il billing è abilitato. Gli extra storage sono calcolati sulla quota aggiunta a pagamento e sulla sua durata, non semplicemente sui byte occupati. Le estensioni gratuite restano identificate separatamente. Configurare tariffe e valore del credito nelle impostazioni e verificare i riepiloghi per intervallo. PayPal usa ordini e catture verificati dal server; gli eventuali rimborsi monetari da regolare non diventano automaticamente crediti.



Accounting e billing: verificare periodo, utente e origine delle misure.

Runtime AI e GPU

install_mta.sh installa i motori AI in runtime versionati sotto /opt/mta/ai-*. Servono al funzionamento: non cancellare manualmente il runtime attivo. La pulizia automatica deve conservare quelli riferiti dalla configurazione. Su EL10 FFmpeg può essere fornito da ffmpeg-free. I gruppi SAM Audio/torchcodec/ xFormers devono corrispondere al backend CPU/CUDA/ROCm; pacchetti CUDA non compatibili non risolvono un ambiente ROCm. Diagnostica mostra installazione, sorgente delle misure, limiti artefatti e dettagli di errore.

In LXC la sola presenza di cgroup o della GPU non garantisce contatori per processo. L'agente host deve avere socket montato, autorizzazioni e compatibilità col kernel/driver. Un contatore a zero o non verificato deve conservare questo stato nei report. Prima del billing verificare origine e politica configurata per le stime. Non dichiarare la piattaforma certificata sulla sola base della CI.

DSP C++, VST3 e rete

Il DSP C++ e i plugin VST3 sono disponibili sul server. Le build RPM/DEB/Docker includono il runtime; i plugin opzionali vengono compilati/installati separatamente nel catalogo gestito. Le licenze restano distinte e vanno riesaminate periodicamente. Un plugin in una sessione live o in un editor aperto non è disinstallabile. L'interfaccia originale remota richiede il backend display; la disponibilità del catalogo non equivale a compatibilità di tutti i plugin.

HAProxy/Ingress deve consentire streaming e WebSocket e non interrompere i job lunghi per timeout di proxy. Il server esegue DSP e AI, mentre il browser riproduce il PCM ricevuto: buffering di rete e tempo DSP sono metriche diverse. Verificare il trasporto reale dopo aggiornamenti del proxy, driver o plugin.

Conservazione, dump e ripristino

La cancellazione normale conserva accounting/billing e le identità non riciclabili; il grace period progetti è configurabile, con default 60 giorni. L'utente riceve il link all'archivio e la scadenza se SMTP è configurato. Negli Archivi utenti cancellati l'admin può cancellare definitivamente i dati con una conferma. Il percorso privacy con sandbox ha conservazione configurabile (default 365 giorni), salvo cancellazione definitiva esplicita. I dump possono selezionare utenti/progetti; un utente include tutti i suoi progetti. Provare un restore in un ambiente isolato prima di usarlo in produzione e mantenere una copia indipendente.

Privacy e termini vengono generati con i dati dell'istanza. Sono necessari Titolare, Paese, email assistenza/reclami, URL pubblica HTTPS e data di efficacia; i campi mancanti facoltativi vengono omessi. Le modifiche ai dati legali notificano gli utenti registrati se SMTP è configurato. I documenti standard richiedono comunque una verifica del gestore per il proprio trattamento e giurisdizione.

22. Checkpoint distribuzione MIDI/GM e moduli AI

Le preferenze editor/sidebar/dispositivi MIDI sono isolate per identità e scritte in modo differito nel database esistente. Patch indipendenti vengono unite in transazione. Non tutte le impostazioni native storiche sono migrate. Un selettore visibile non dimostra compatibilità hardware.

GM richiede una libreria FluidSynth dinamica e il banco FluidR3 GM 3.1 verificato (148.398.306 byte). Override operatore: MTA_FLUIDSYNTH_LIBRARY e MTA_GM_SOUNDFONT_DIR. Il checksum del banco viene verificato prima dell'uso. FluidSynth e banco hanno licenze LGPL/MIT separate: conservarne gli avvisi. Binari e dati del banco non sono inclusi nel sorgente.

Da Modelli Studio accettare esplicitamente le licenze collegate di Basic Pitch e installare il runtime separato Python 3.11/3.12 ONNX CPU. Un test reale di inferenza e il checksum precedono l'attivazione. Gli amministratori possono ripetere il test, abilitare/disabilitare e disinstallare il modello; i job attivi ne impediscono la rimozione. Sono configurabili timeout (default 3600 s) e conservazione intermedi (default sette giorni). La telemetria ONNX è disattivata prima della inizializzazione. Restano coda, accounting e permessi esistenti. Non viene dichiarata compatibilità ROCm/CUDA o consumo GPU per Basic Pitch. Modelli vocali autorizzati restano indisponibili.

YourMT3 noPS è un backend CPU aggiuntivo installabile dall'operatore per trascrivere mix in parti GM/ percussioni previste. Una prova reale di pianoforte è passata dalla coda ed è stata salvata come MIDI modificabile. Non è un benchmark di accuratezza su dieci strumenti. La velocity è generata; non viene indicata confidenza calibrata. Installazione, hash e licenze: docs/YOURMT3_RUNTIME_R210.md. Restano canto sintetico e riconciliazione multimodello.

Nelle app native, Dispositivi audio / ASIO / routing seleziona API, dispositivi fisici, buffer e matrici master stereo/ingresso. Il callback compilato non chiama Python. Uno stream duplex supporta riproduzione e registrazione stereo. Restano certificazione ASIO fisica, bus hardware indipendenti per traccia e bundle di sorgenti corrispondenti per ridistribuire ASIO. Modelli Studio usa API locali dedicate; i workflow remoti condividono il token e recuperano risultati senza doppi addebiti.



Workflow Studio e capacità MIDI effettive

Appendici tecniche

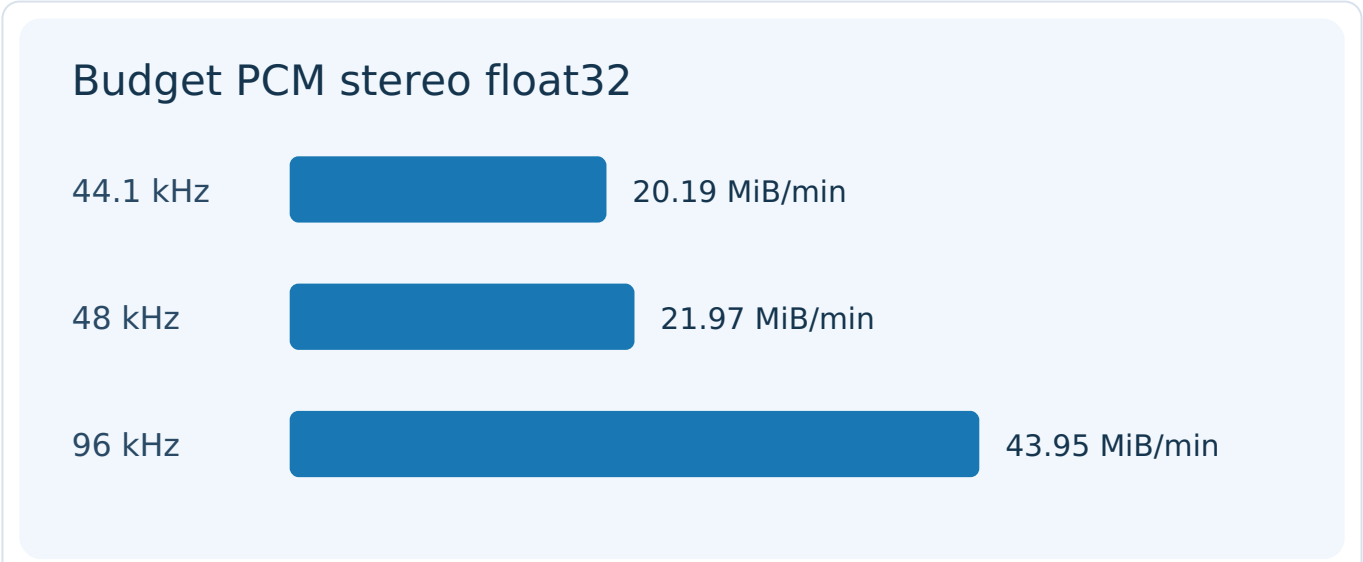
Stato documentato: 0.3.0-r210. Le indicazioni di dimensionamento sono stime operative, non requisiti minimi certificati. I risultati locali non certificano dispositivi, plugin o server di produzione.

A. Tipi di applicazione e funzionalità

Tipo	Audio / DSP	AI	Plugin / MIDI
Web + server	Browser Web Audio; C++ DSP sul server	Coda globale server; token per client remoti	VST3 Linux sul server, editor remoto; Web MIDI ove supportato
Windows desktop	Motore corrente o C++; trasporto su dispositivo	Motori locali oppure server remoto con token	VST3 Windows compatibili; MIDI nativo
macOS desktop	Intel / Apple Silicon; corrente o C++	Modelli locali o server; supporto acceleratore dipendente dal motore	Plugin macOS della stessa architettura; MIDI nativo
Linux desktop	Motore corrente o C++; dispositivo locale	CPU / CUDA / ROCm secondo runtime e hardware	VST3 Linux, ALSA/MIDI secondo disponibilità
Android / iOS / iPadOS	Shell mobile, audio del browser/WebView	Server; eventuali percorsi on-device solo con modello/ runtime compatibile	Plugin sul server; MIDI browser secondo piattaforma

B. Risorse e dimensionamento

Profilo	CPU / RAM	Disco / rete	Note
Browser	4-8 GiB RAM del dispositivo: base di pianificazione	HTTPS; rete stabile per streaming e editor remoti	Cache PCM 256 MiB; nessuna garanzia su WebView obsoleti
Desktop / DSP	4 core / 8 GiB: punto di partenza, 16+ GiB per sessioni dense	SSD consigliato; spazio per sorgenti, export e modelli	Plugin, frequenza e canali determinano il carico reale
Server editor	4 vCPU / 8 GiB per test contenuti; dimensionare per concorrenza	Volume persistente, backup, banda per sessioni simultanee	Non è una capacità garantita per numero di utenti
Server / desktop AI	16-32+ GiB RAM come budget iniziale; dipende dal modello	Pesi e runtime separati: diversi GiB, talvolta decine	VRAM/unified memory e precisione dipendono dal modello; verificare ciascun motore
Mobile	Memoria/CPU del browser; evitare progetti enormi	Wi-Fi stabile consigliato per audio e plugin remoti	Il DSP server non diventa locale sul telefono



Memoria PCM teorica per minuto, stereo float32 (esclusi grafi, modelli e cache duplicate).

```
byte PCM = secondi × frequenza × canali × 4
```

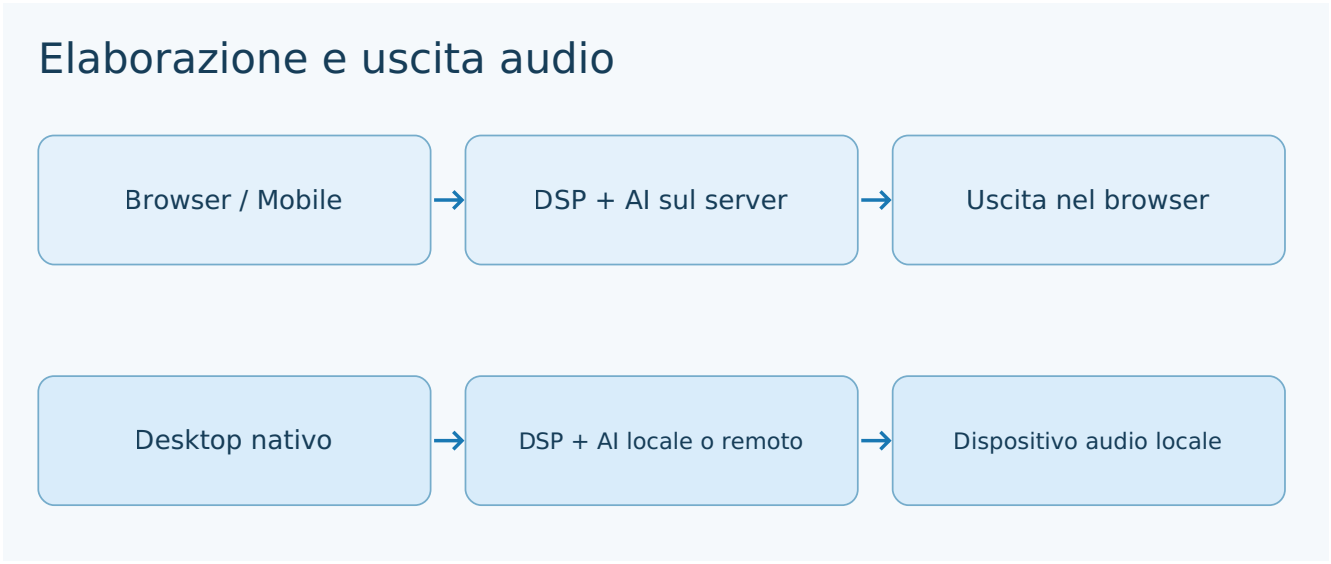
C. Limiti e comportamento

Area	Limite / comportamento
Progetto	44.1 / 48 / 96 kHz; frequenza fissa dopo la prima traccia. Timeline fino a 24 h.
Metronomo	Standard, zone con marker, adattivo. Pre-roll 0-64 battute; BPM griglia 20-400, anche decimali.
FX	Fino a 16 insert per catena; dipendenze e compatibilità VST3 da verificare per plugin.
MIDI	SMF0/1/2 fino a 8 MiB; base tempi PPQ e SMPTE. UMP/MIDI 2.0 secondo driver/provider, non garantito per ogni dispositivo.
AI	Coda globale; limiti artefatti e timeout configurabili. Un modello installato non garantisce GPU compatibile.
Streaming C++	Buffer adattivo 0,35-1,2 s più blocco; non equivale a monitoring a latenza professionale garantita.
Storage / billing	Default nuovi utenti 5 GiB; admin senza quota per default. Quote/crediti dipendono dal piano e dai permessi.

D. Prestazioni osservate e validazione

Misure del precedente checkpoint locale: Chromium, otto tracce stereo da 30 s; carico e macchina di prova non equivalgono al server di produzione. I rapporti originali restano nella cartella validation del progetto.

Prova	Risultato osservato	Fonte locale
C++ playback	Avvio 1,239 s; risposta fader 0,538 s; 0 underrun	r210-continuous-pcm-browser.json
C++ jitter / UI stalls	Ritardi rete fino a 180 ms, cinque stalli UI da 450 ms: 0 underrun	r210-continuous-pcm-stress.json
Motore corrente	Avvio 2,198 s; riavvio con cache 0,265 s	r210-continuity-current-engine.json



Dove avvengono elaborazione e uscita audio.

Certificazione ancora dipendente da prove sulla piattaforma reale: scheda audio, driver MIDI, plugin terzi, GPU e proxy. I consumi accettati sono contabilizzati; origine e validità restano nell'audit; il consumo zero non prova assenza di utilizzo GPU.

Strumenti GM, canto e licenze r210

Da AI Studio, aprire Esecuzione e Modelli Studio per installare esplicitamente il banco GM FluidR3 (MIT) e le voci CC0 OddVoices, con stato e avanzamento. OddVoices produce canto concatenativo su CPU: non è un backend neurale DiffSinger. Dalla scheda Cantanti si possono sintetizzare parti MIDI monofoniche come vocalizzi o sillabe inglesi, salvando insieme audio e MIDI. Italiano/francese e conversione timbrica non sono dichiarati disponibili.

Audio-to-MIDI in modalità Accurata usa YourMT3 e Basic Pitch, conserva le ipotesi e permette di importare alternative MIDI modificabili. Non attribuisce una falsa probabilità alle note né garantisce accuratezza su dieci strumenti arbitrari. La modalità Massima accuratezza non è attiva.

Le app native e l'elaborazione locale sono gratuite. Gli eventuali addebiti del server riguardano CPU/GPU/disco come rimborso hardware senza lucro; le licenze dei modelli restano applicabili. Le policy, i componenti disabilitati e le condizioni di attivazione manuale sono documentati in docs/MUSIC_ASSETS_AND_LICENSES_R210.md, incluso nel package. Il nuovo script verify_music_hardware.py prepara il collaudo ASIO/ROCm; l'inventario non costituisce certificazione.



Installer e stato delle voci: schermata reale dopo le prove CPU, con policy licenze collassabile.

23. SAM Audio: accesso e diagnosi del servizio

I checkpoint SAM Audio sono ad accesso controllato. Con lo stesso account Hugging Face, aprire le pagine SAM Audio Base e SAM Audio Large, accettare le condizioni e attendere l’approvazione ove richiesta. Il token personale deve consentire la lettura dei repository gated autorizzati. Un errore `GatedRepoError` HTTP 401/403 indica un problema di autenticazione o autorizzazione, non prova un guasto GPU.

Il login deve essere visibile all’utente e alla cache usati dal processo del server, non soltanto all’utente amministratore che apre una shell. Per leggere impostazioni effettive senza stampare i valori dei token:

```
sudo python3 - <<'PYCODE'
import os, pwd, subprocess
pid = int(subprocess.check_output(
    ['systemctl', 'show', 'mta-server.service',
     '-p', 'MainPID', '--value'], text=True))
if pid <= 0:
    raise SystemExit('mta-server.service is not running')
with open(f'/proc/{pid}/environ', 'rb') as stream:
    entries = stream.read().split(b'\0')
env = dict(item.split(b'=', 1) for item in entries if b'=' in item)
print('User:', pwd.getpwuid(os.stat(f'/proc/{pid}').st_uid).pw_name)
for key in (b'HOME', b'XDG_CACHE_HOME', b'HF_HOME', b'HF_TOKEN_PATH'):
    print(key.decode(), ':', os.fsdecode(env.get(key, b'(not set)')))
for key in (b'HF_TOKEN', b'HUGGING_FACE_HUB_TOKEN'):
    print(key.decode(), ':', 'set' if env.get(key) else 'not set')
PYCODE
```

Nel server verificato il servizio usa `root`, `HF_HOME=/var/lib/mta/models/huggingface`, nessun `HF_TOKEN_PATH` personalizzato e nessun token nell'ambiente. Il Python SAM rilevato era `/opt/mta/ai-7f176167db5671ae/sam/bin/python`. Questo percorso è specifico dell'installazione: individuare quello corrente prima di riutilizzarlo. Con tali valori il login e la prova dei soli file di configurazione sono:

```
SAM_HF=/opt/mta/ai-7f176167db5671ae/sam/bin/hf
sudo env HF_HOME=/var/lib/mta/models/huggingface \
  "$SAM_HF" auth login
sudo env HF_HOME=/var/lib/mta/models/huggingface \
  "$SAM_HF" auth whoami
sudo env HF_HOME=/var/lib/mta/models/huggingface \
  "$SAM_HF" download facebook/sam-audio-base config.json
sudo env HF_HOME=/var/lib/mta/models/huggingface \
  "$SAM_HF" download facebook/sam-audio-large config.json
```

Per un servizio non root eseguire i comandi come l'utente effettivo, con il suo `HF_HOME` e l'eventuale `HF_TOKEN_PATH`. Un `HF_TOKEN` nell'ambiente ha precedenza sul token salvato; verificare anche se è configurato `HUGGING_FACE_HUB_TOKEN`, il cui comportamento dipende dalla versione. Le verifiche scaricano solo `config.json`; la prima inferenza può richiedere i pesi completi e spazio sufficiente.

> **Credenziali:** non incollare token in chat o nei ticket e non stamparli nei log. Conservare le credenziali con permessi adeguati all'utente del servizio. L'accesso al modello e l'effettiva compatibilità GPU restano verifiche distinte.

Se l'errore persiste, controllare i log con `sudo journalctl -u mta-server.service --no-pager` e condividere solo le righe necessarie dopo aver oscurato credenziali e dati personali. I bundle diagnostici strutturati possono indicare la fase senza includere l'eccezione originale.